

Predicting Customer Churn in an Audiobook App Using Machine Learning

Introduction

In today's competitive digital landscape, user retention is critical for the long-term success of subscription-based platforms. Audiobook apps, in particular, rely heavily on engaged and returning customers to sustain growth. One of the key challenges in this space is predicting customer churn—identifying which users are likely to stop using the app before they actually do.

This project applies supervised machine learning classification techniques to a dataset from an audiobook application. The main objective is to develop a model that can predict whether a user is likely to churn, using their activity and purchase behavior data. In our context, churn is defined as not having made a purchase in the last 6 months of the observation period.

Problem Statement

Given historical user interaction data, we aim to classify users into two categories:

- Churned (0): The user has not made a purchase in the final 6 months of data.
- Active (1): The user has made at least one purchase in the final 6 months.

This binary classification problem will allow us to explore various ML models, evaluate their performance, and ultimately choose the one that best identifies at-risk customers so that proactive retention strategies can be implemented.

Dataset Overview

The dataset includes anonymized records of user behavior on the audiobook platform, such as:

- Number of minutes listened
- Book completion percentage
- Latest engagement in materials past the purchase date
- Number of support request claims made
- Average book cost

These features will serve as inputs to train our machine learning models to predict churn

outcomes. We'll also perform data cleaning, feature engineering, model evaluation, and performance tuning to ensure the final model is both accurate and interpretable.

```
In [169]: import kagglehub
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools.tools import add_constant
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import HistGradientBoostingClassifier, RandomForestClassifier
from sklearn.svm import SVC
from imblearn.over_sampling import SMOTE
import numpy as np
import joblib

# Download latest version
path = kagglehub.dataset_download("faressayah/audiobook-app-data")

print("Path to dataset files:", path)
```

Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.10), please consider upgrading to the latest version (0.3.11).

Path to dataset files: /Users/trentonsmiley/.cache/kagglehub/datasets/faressayah/audiobook-app-data/versions/2

```
In [83]: df = pd.read_csv(path + "/audiobook_data_2.csv")
```

```
In [84]: df.info()
```

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 14084 entries, 0 to 14083

Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
0	Unnamed: 0	14084 non-null	int64
1	Book_length(mins)_overall	14084 non-null	float64
2	Book_length(mins)_avg	14084 non-null	int64
3	Price_overall	14084 non-null	float64
4	Price_avg	14084 non-null	float64
5	Review	14084 non-null	int64
6	Review10/10	14084 non-null	float64
7	Completion	14084 non-null	float64
8	Minutes_listened	14084 non-null	float64
9	Support_Request	14084 non-null	int64
10	Last_Visited_mins_Purchase_date	14084 non-null	int64
11	Target	14084 non-null	int64

dtypes: float64(6), int64(6)

memory usage: 1.3 MB

Column Renaming and Cleanup

To improve clarity and consistency across our dataset, we perform a column renaming operation using a predefined mapping. This step aligns the column names with more descriptive, readable formats, making the data easier to interpret and work with throughout the project.

Notably, there is a discrepancy between the column names and the data descriptions:

The columns "Book_length(mins)_overall" and "Book_length(mins)_avg" appear to be switched in meaning based on the data structure and documentation. We resolve this by intentionally flipping their labels to reflect their true meaning in the renamed columns. Additionally, we remove one unnecessary column:

- "User_Id": already captured in the index or not relevant to model training.

This results in a cleaner, more intuitive dataset for analysis and modeling.

```
In [85]: column_mapper = {  
    "Unnamed: 0" : "User_Id",  
    "Book_length(mins)_overall" : "Book_Length_Average", # There is a discre  
    "Book_length(mins)_avg" : "Book_Length_Overall", # these two columns wer  
    "Price_overall" : "Book_Price_Overall",  
    "Price_avg" : "Book_Price_Average",  
    "Review" : "Left_Review",  
    "Completion" : "Book_Completion_Percentage",  
    "Support_Request" : "Number_Support_Requests",  
    "Last_Visited_mins_Purchase_date" : "Lastest_Engagment_Past_Purchase",  
}  
df = df.rename(column_mapper, axis=1).drop(["User_Id"], axis=1)
```

```
In [56]: df.describe()
```

```
Out[56]:
```

	Book_Length_Average	Book_Length_Overall	Book_Price_Overall	Book_Price_A
count	14084.000000	14084.000000	14084.000000	14084.0
mean	1591.281685	1678.608634	7.103791	7.5
std	504.340663	654.838599	4.931673	5.1
min	216.000000	216.000000	3.860000	3.8
25%	1188.000000	1188.000000	5.330000	5.3
50%	1620.000000	1620.000000	5.950000	6.0
75%	2160.000000	2160.000000	8.000000	8.0
max	2160.000000	7020.000000	130.940000	130.9

Exploring Feature Relationships with a Correlation Heatmap

To understand how our numerical features relate to one another, we compute a correlation matrix and visualize it using a heatmap. This helps us identify:

- Strong linear relationships between variables (positive or negative),
- Potential multicollinearity (highly correlated predictors),
- And features that may have a meaningful connection to our target variable (Churn).

We calculate pairwise Pearson correlations between all numeric columns using `df.corr(numeric_only=True)`, and plot the result with Seaborn's heatmap.

The heatmap uses the coolwarm color palette:

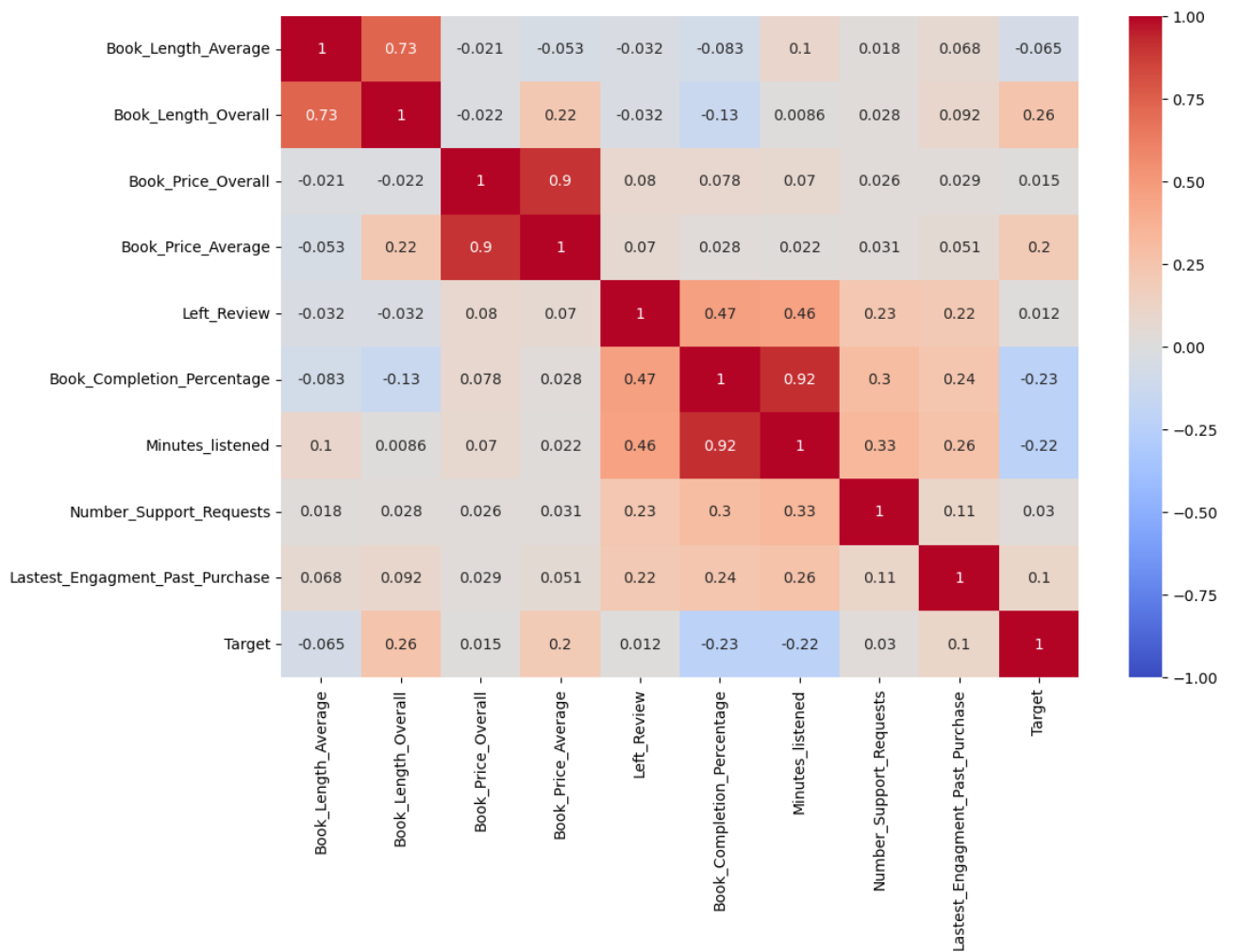
- Red values indicate a strong positive correlation,
- Blue values indicate a strong negative correlation,
- Values near zero (white) suggest little or no linear relationship.

This visualization is a key step in guiding our feature selection and engineering process.

```
In [57]: corr = df.corr(numeric_only=True)

fig = plt.figure(figsize=(12, 8))
sns.heatmap(corr, cmap="coolwarm", annot=True, vmin=-1, vmax=1)
```

```
Out[57]: <Axes: >
```



Removing Multicollinear Features

From the correlation heatmap above, we observe that some features exhibit strong linear relationships with one another, indicating multicollinearity. While correlated features are not inherently bad, high multicollinearity can reduce model interpretability, inflate variance in coefficient estimates (for linear models), and introduce redundancy in the feature space.

To address this, we removed the following columns from consideration in our model:

- Book_Length_Average: Strongly correlated with Book_Length_Overall ($r = 0.73$).
- Book_Price_Overall: Highly correlated with Book_Price_Average ($r = 0.90$).
- Minutes_listened: Extremely collinear with Book_Completion_Percentage ($r = 0.92$).

By dropping these redundant features when creating the model, we:

- Reduce dimensionality without significant information loss,
- Improve model training stability,
- Ensure cleaner feature importance metrics and interpretations.

The retained features in each case are either more granular or more intuitively meaningful for modeling churn behavior.

```
In [58]: reduced_df = df.drop(["Book_Length_Average", "Book_Price_Overall", "Minutes_
```

✓ Verifying Multicollinearity Removal with VIF

To ensure that our dataset no longer suffers from multicollinearity, we conducted a Variance Inflation Factor (VIF) analysis. VIF measures how much the variance of a regression coefficient is inflated due to linear relationships with other predictors. High VIF values (typically > 5 or 10) suggest that a feature is highly collinear and could negatively impact model stability or interpretability.

We calculated the VIF for all remaining numeric features after our initial round of correlation-based feature elimination. The results are as follows:

```
In [59]: X = reduced_df.select_dtypes(include='number') # use only numeric columns
X = add_constant(X) # required for intercept

vif_data = pd.DataFrame()
vif_data["feature"] = X.columns
vif_data["VIF"] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
vif_data
```

```
Out[59]:
```

	feature	VIF
0	const	9.057528
1	Book_Length_Overall	1.130037
2	Book_Price_Average	1.086236
3	Left_Review	1.330723
4	Book_Completion_Percentage	1.531687
5	Number_Support_Requests	1.125998
6	Lastest_Engagment_Past_Purchase	1.110252
7	Target	1.190157

These values are all well below the threshold of concern ($VIF < 5$), confirming that our dataset is now free from significant multicollinearity. This means:

- Each feature contributes unique information to the model,
- Model coefficients will be more stable,

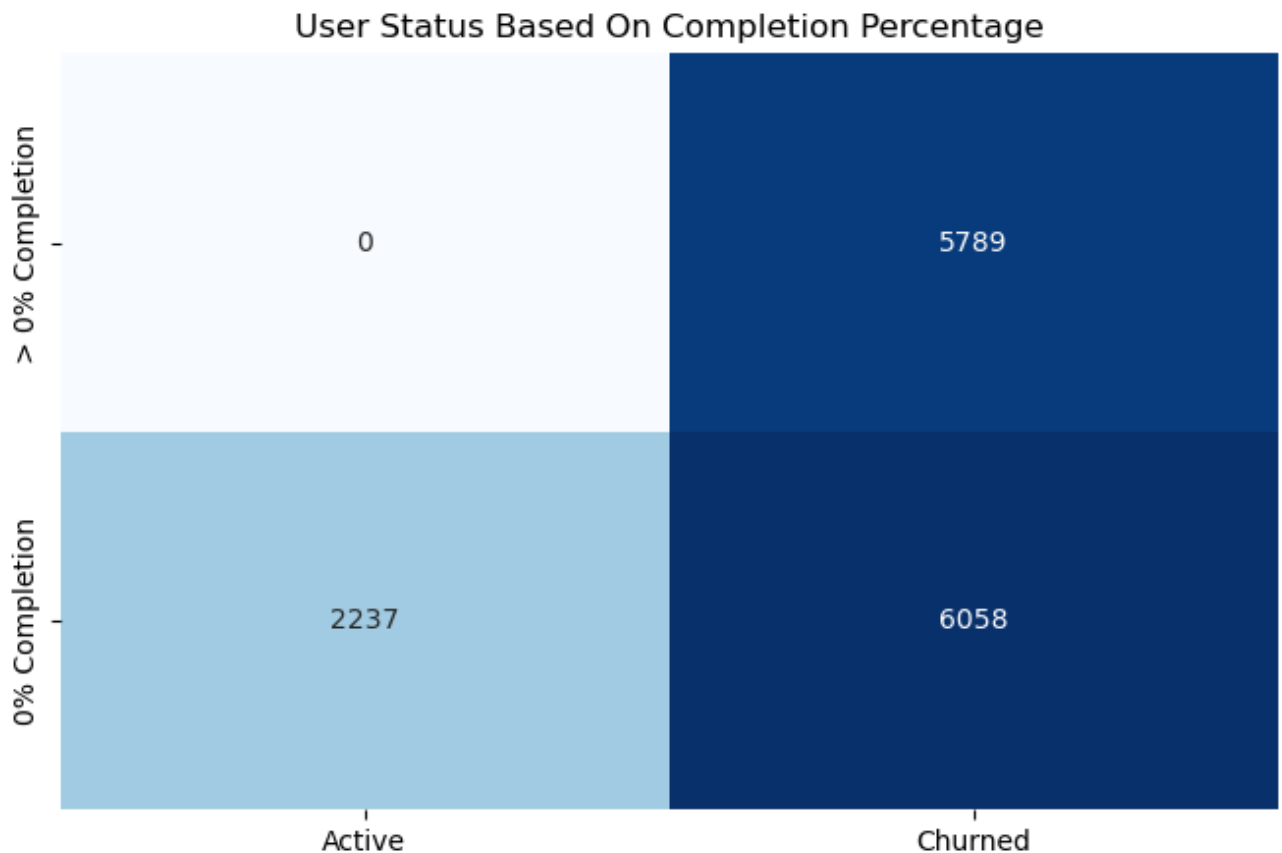
- And the interpretability of feature importance improves.

With this confirmation, we can confidently proceed to model training using a clean and optimized set of features.

```
In [60]: number_completion_target1 = len(reduced_df[(reduced_df["Book_Completion_Percentage"] > 0)])
number_completion_target0 = len(reduced_df[(reduced_df["Book_Completion_Percentage"] == 0)])
number_notcompletion_target1 = len(reduced_df[(reduced_df["Book_Completion_Percentage"] > 0)])
number_notcompletion_target0 = len(reduced_df[(reduced_df["Book_Completion_Percentage"] == 0)])

data = [[number_completion_target1, number_completion_target0],
        [number_notcompletion_target1, number_notcompletion_target0]]
data = pd.DataFrame(data, index=['> 0% Completion', '0% Completion'], columns=['Active', 'Churned'])

fig = plt.figure(figsize=(8, 5))
sns.heatmap(data, annot=True, fmt="d", cmap="Blues", cbar=False)
title = plt.title("User Status Based On Completion Percentage")
```



🎯 Completion as Closure: A Hidden Churn Signal

We now examine the starkest contrast — users who completed nothing (0%) vs. users who completed anything (>0%).

The resulting heatmap shows a compelling pattern:

	Active	Churned
> 0% Completion	0	5,789
0% Completion	2,237	6,058

Key Insights:

- Every user who completed more than 0% of content churned. The top-left cell of the heatmap shows zero active users among those who completed any content — an alarming pattern at face value.
- A large number of churned users (6,058) had 0% completion, reinforcing that lack of engagement is still a strong churn signal.
- Interestingly, 2,237 users with 0% completion remained active, suggesting that some users stay on the platform while still exploring, browsing, or sampling without finishing any content.
-  **Counterintuitive Insight:** Users who complete books may actually be at higher risk of churn. This suggests that some users find a single book they're interested in, complete it, and then leave the platform feeling satisfied, without the need to return. In this case, completion may represent fulfillment rather than ongoing engagement — a "mission accomplished" behavior rather than a sign of loyalty.

Conclusion

This analysis reveals a surprising and counterintuitive dynamic between audiobook completion and user churn. While we might expect content completion to reflect high engagement and retention, our data tells a different story.

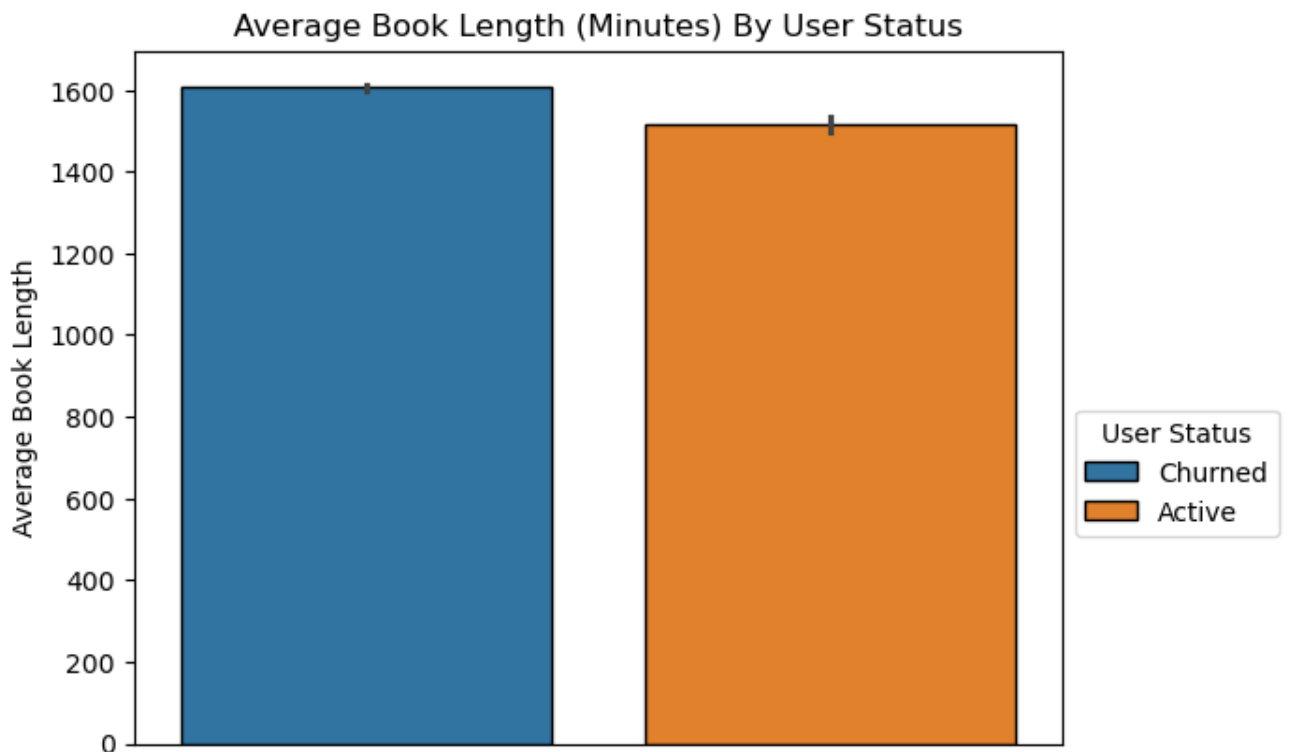
Users who completed more than 0% of any audiobook all eventually churned, suggesting that completion may signal closure rather than loyalty. These users may join the platform with a specific book in mind, finish it, and then feel no need to return — a "one-and-done" usage pattern.

Meanwhile, the only active users in our dataset were those who had never completed a single book. This may represent users who are still browsing, sampling, or just starting their engagement journey. These users haven't yet found a fulfilling experience — and that might be precisely what keeps them coming back.

Rather than viewing completion as a success metric, we should consider it a trigger point for targeted interventions:

Recommend new or similar titles right after completion, Introduce follow-up content pathways, Offer incentives for continued exploration. In short, retention isn't just about helping users finish — it's about giving them a reason to continue.

```
In [75]: plot = sns.barplot(df, x="Target", y="Book_Length_Average", hue="Target", ec=
plt.ylabel("Average Book Length")
plt.xlabel("")
plt.xticks([])
handles, labels = plot.get_legend_handles_labels()
legend = plt.legend(handles = handles, labels=["Churned", "Active"], title="
title = plt.title("Average Book Length (Minutes) By User Status")
```



Average Book Length and Churn: No Clear Separation

To explore whether book length preferences differ between user groups, we compared the average book length (in minutes) for users who churned versus those who remained active.

The bar plot above shows the mean book length for each group, with error bars representing standard error.

Key Observation:

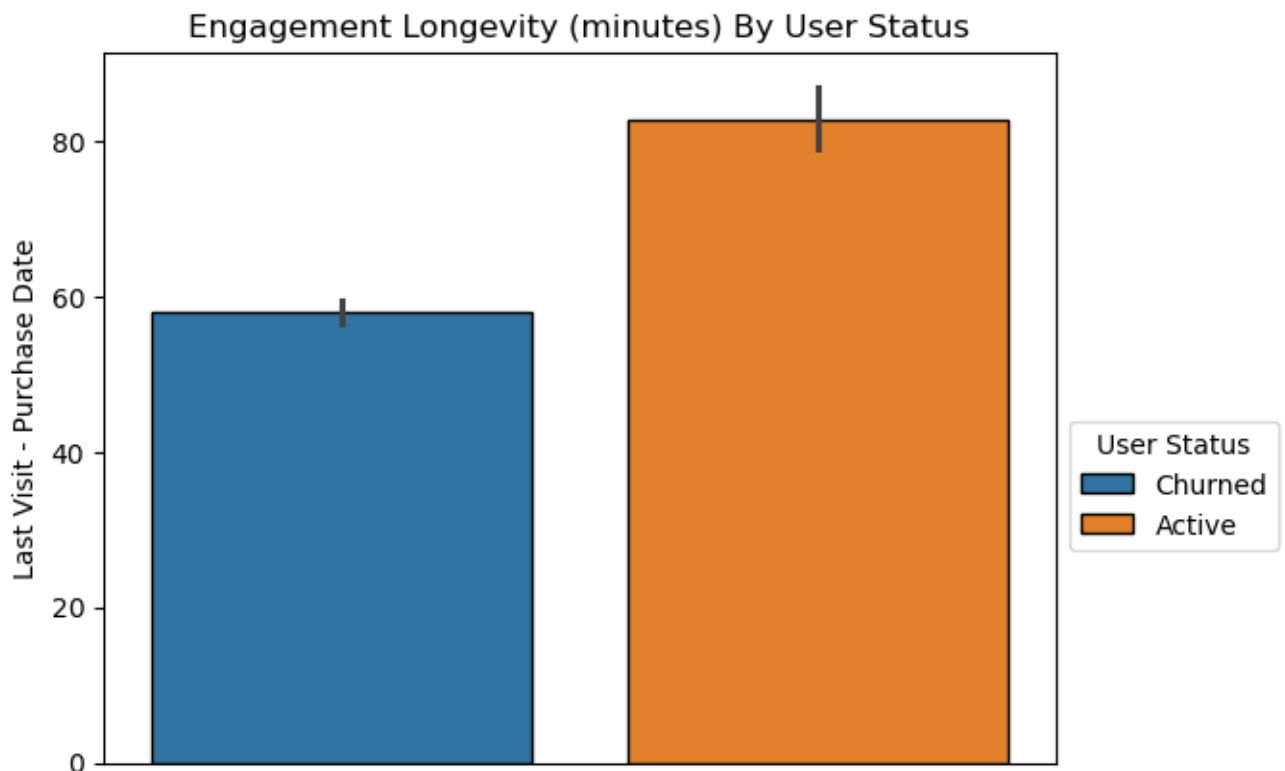
Churned users had a slightly higher average book length (~1,600 minutes) compared to

active users (~1,500 minutes). However, the difference is relatively small and well within the range of expected variation. The error bars overlap significantly, indicating no statistically meaningful separation between the two groups based on book length alone.

✓ Conclusion:

Average book length does not appear to be a useful feature for predicting churn. Users consume similarly sized content regardless of whether they continue using the app or not. This suggests that content duration is not a driver of engagement in this dataset, and other behavioral features (like completion or interaction patterns) may be more informative for churn modeling.

```
In [80]: plot = sns.barplot(df, x="Target", y="Lastest_Engagment_Past_Purchase", hue=
plt.ylabel("Last Visit - Purchase Date")
plt.xlabel("")
plt.xticks([])
handles, labels = plot.get_legend_handles_labels()
legend = plt.legend(handles = handles, labels=["Churned", "Active"], title="
title = plt.title("Engagement Longevity (minutes) By User Status")
```



🔄 Long-Term Engagement After Purchase Predicts Retention

This bar plot compares the engagement duration after a user's first purchase —

calculated as the time (in minutes) between their first purchase and their most recent visit — for churned vs. active users.

Key Observation:

- Active users show significantly longer engagement durations after their initial purchase (~83 minutes) compared to churned users (~58 minutes).
- This indicates that users who continue to return to the platform well after making a purchase are much more likely to remain active.
- In contrast, churned users tend to show shorter or single-day engagement windows, or in some cases, no engagement at all after purchase (value = 0).

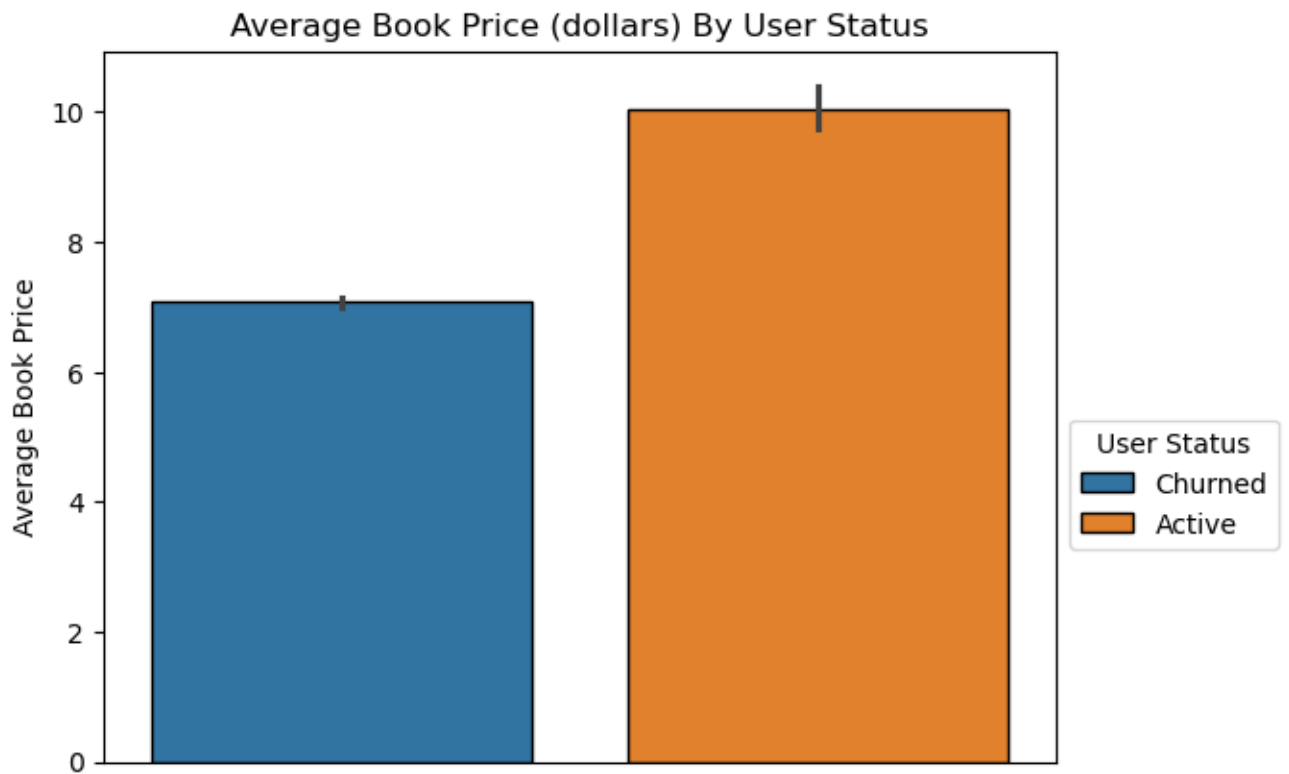
Conclusion:

This feature serves as a strong retention signal: The longer a user continues to visit the app after making a purchase, the more likely they are to stay.

It captures user follow-through and the development of an ongoing habit — which are both key to predicting and promoting long-term retention. This insight could be used to:

- Trigger alerts for low-engagement users soon after purchase,
- Launch automated campaigns that encourage users to revisit the platform days or weeks after buying,
- Identify your most loyal and high-LTV customers based on behavioral longevity.

```
In [92]: sns.barplot(df, x="Target", y="Book_Price_Average", hue="Target", ec="black")
plt.ylabel("Average Book Price")
plt.xlabel("")
plt.xticks([])
handles, labels = plot.get_legend_handles_labels()
legend = plt.legend(handles = handles, labels=["Churned", "Active"], title="")
title = plt.title("Average Book Price (dollars) By User Status")
```



💰 Book Price and Retention: Higher Spend, Higher Loyalty?

This bar plot compares the average price paid per audiobook by users who churned versus those who remained active.

🔍 Key Observation:

- Active users spent significantly more per book (~10) compared to churned users (~7) on average.

The difference is not only visually apparent but also statistically meaningful, as shown by the non-overlapping error bars.

- This suggests that users who invest more in individual purchases may place higher value on the platform experience and are thus more likely to return.

✅ Conclusion:

Book price appears to be a meaningful signal for user retention. Higher-spending users are more likely to stay active, potentially because they:

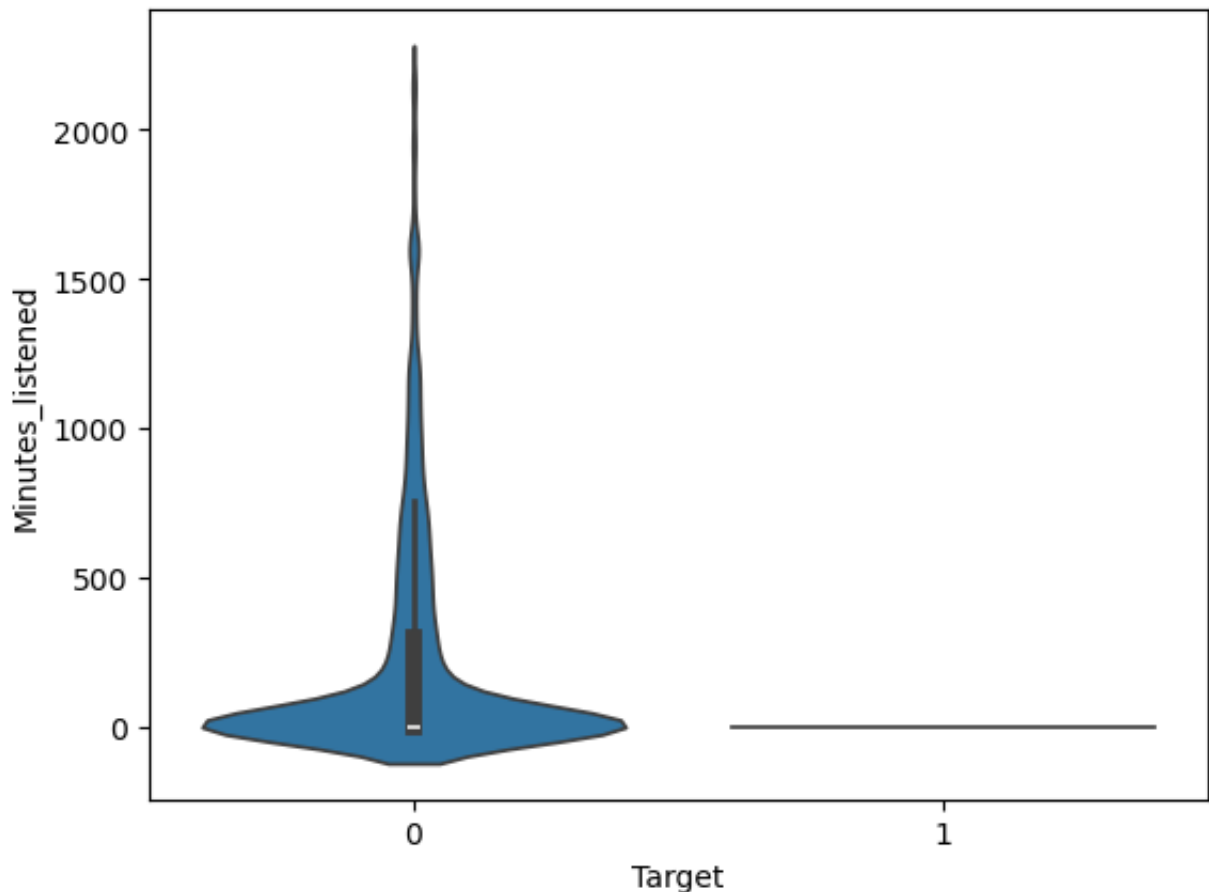
- Are more committed to their purchases,
- View the platform as a source of premium content,
- Or feel more invested and compelled to follow through.

This insight could inform monetization and retention strategies such as:

- Offering premium content bundles to engaged users,
- Targeting lower-price purchasers with engagement campaigns,
- Encouraging upsell opportunities early in a user's lifecycle.

```
In [93]: sns.violinplot(df, x="Target", y="Minutes_listened")
```

```
Out[93]: <Axes: xlabel='Target', ylabel='Minutes_listened'>
```



Revisiting the “One-and-Done” Pattern Through Listening Time

The violin plot above visualizes total minutes listened for users labeled as churned (Target = 0) versus active (Target = 1). Surprisingly, we observe that:

- Churned users show a wide distribution of listening time — from brief sessions to deep, prolonged engagement (well over 2,000 minutes).

- Active users, on the other hand, show virtually no listening activity, with their entire distribution collapsed near zero.

Linking It Back: Completion ≠ Retention

This mirrors the earlier findings from:

- The completion percentage heatmap, where users who completed any amount of a book all eventually churned,
- The review data, where some users likely gave a high score and then left,
- The long-term engagement plot, where shorter engagement windows predicted churn.

Together, these findings reveal a consistent behavioral pattern:

Many users engage deeply with the platform once — listening, completing, and possibly reviewing — and then never return, even if they technically remain "active" for a while based on purchase windows.

This "one-and-done" dynamic flips traditional assumptions:

Engagement (e.g., completion, listening time, even high ratings) doesn't necessarily indicate loyalty. Lack of engagement, paradoxically, might reflect users still in the discovery phase — and thus more likely to stick around.

Conclusion:

Listening time, like completion rate, is a closure signal more than a retention signal in this dataset. Users who've already extracted value may have no reason to return, whereas those who haven't engaged much might still be exploring — and still convertible.

This reinforces the need to rethink what engagement really means in churn models — and to watch for "mission-accomplished" signals as early warnings of silent attrition.

```
In [129... X = reduced_df.drop("Target", axis=1)
y = reduced_df["Target"]
sampler = SMOTE(sampling_strategy = {1 : y[y == 0].count() // 2})
resampled_X, resampled_y = sampler.fit_resample(X, y)

scaler = StandardScaler()
scaled_X = scaler.fit_transform(resampled_X, resampled_y)
scaled_X.shape
```

```
Out[129... (17770, 6)
```

Addressing Class Imbalance with Targeted Oversampling

To improve model performance and reduce bias caused by class imbalance, we applied a controlled oversampling strategy using SMOTE (Synthetic Minority Over-sampling Technique). Rather than fully balancing the dataset, we adopted a hybrid approach that brings the minority class closer to the majority without making them equal.

What We Did:

```
sampler = SMOTE(sampling_strategy={1: y[y == 0].count() // 2})
resampled_X, resampled_y = sampler.fit_resample(X, y)
```

- We created synthetic examples for the minority class (Target = 1, active users).
- Instead of matching the count of churned users (Target = 0), we increased the minority class to half the size of the majority class.
- This results in a 2:1 class ratio (churned:active), maintaining some real-world imbalance while helping the model learn meaningful patterns in the minority group.

Why This Hybrid Strategy?

- Fully balancing the dataset can distort the natural class distribution and potentially lead to overfitting or unrealistic decision boundaries.
- A 2:1 ratio strikes a balance, making the model more sensitive to the minority class while still respecting the dominant real-world behavior.
- During training, we reinforce this emphasis by setting `class_weight={0: 1, 1: 2}` in our models, ensuring misclassifying active users carries double the penalty of misclassifying churned users.

Feature Scaling for Sensitive Models

Before training models like LogisticRegression and SVC, we applied feature standardization using StandardScaler:

```
scaler = StandardScaler()
resampled_X = scaler.fit_transform(resampled_X, resampled_y)
```

Why this matters:

- Models like logistic regression and support vector machines are sensitive to feature scales, especially when features have varying units or ranges.

- Standardization transforms the data to have zero mean and unit variance, improving model convergence and the effectiveness of regularization.

⚠ Note: Tree-based models like RandomForestClassifier and HistGradientBoostingClassifier do not require scaling, but applying it universally at this stage ensures consistency across models.

```
In [130... X_train, X_test, y_train, y_test = train_test_split(scaled_X, resampled_y)
y_train, y_test = np.array(y_train), np.array(y_test)
class_weight = {0:1, 1:2}
logReg = LogisticRegression(class_weight=class_weight)
randForest = RandomForestClassifier(class_weight=class_weight)
svc = SVC(class_weight=class_weight)
hgbc = HistGradientBoostingClassifier(class_weight=class_weight)
```

Model Setup: Baseline and Tree-Based Classifiers with Class Weights

To begin evaluating our predictive models for churn, we prepared a training and testing split using the resampled dataset. The resampling strategy was previously chosen to create a 2:1 class imbalance, which better reflects the real-world churn distribution while still addressing minority underrepresentation.

Train/Test Split

We used `train_test_split()` to randomly divide the resampled data:

```
X_train, X_test, y_train, y_test = train_test_split(resampled_X,
resampled_y)
```

This ensures that models are trained and evaluated on separate data, providing an unbiased measure of generalization.

Class Weighting Strategy

To reinforce the importance of correctly predicting the minority class (Target = 1, active users), we set a custom class weighting scheme:

```
class_weight = {0: 1, 1: 2}
```

This doubles the penalty for misclassifying an active user, aligning with our hybrid resampling strategy.

Models Prepared for Evaluation:

We configured four supervised learning models with the specified `class_weight`:

```
logReg = LogisticRegression(class_weight=class_weight)
randForest = RandomForestClassifier(class_weight=class_weight)
svc = SVC(class_weight=class_weight)
hgbc = HistGradientBoostingClassifier(class_weight=class_weight)
```

Models:

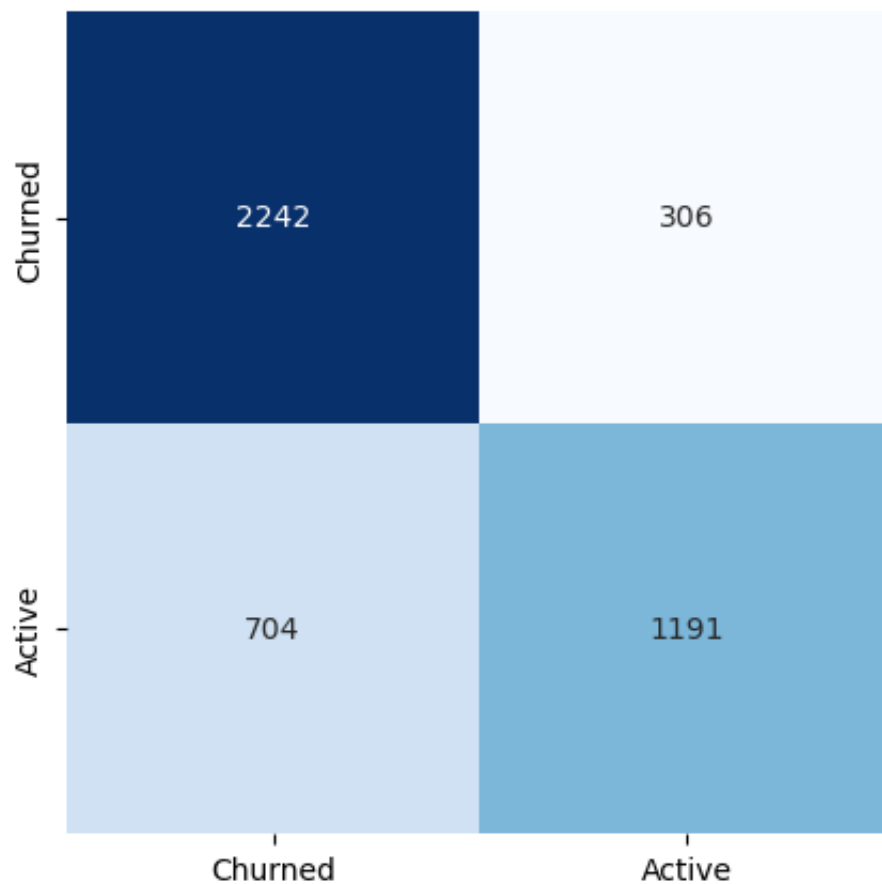
- **LogisticRegression**: A linear baseline classifier.
- **RandomForestClassifier**: A robust ensemble tree model.
- **SVC**: A margin-based model for capturing complex boundaries (note: may require scaling and is slower on large data).
- **HistGradientBoostingClassifier**: A powerful, histogram-optimized boosting model for tabular data.

Each model will be trained and evaluated using classification metrics such as accuracy, precision, recall, F1-score, and confusion matrix to compare performance — with special attention to how well each model handles the minority (active) class.

```
In [149... logReg.fit(X_train, y_train)
y_pred = logReg.predict(X_test)

matrix = confusion_matrix(y_pred, y_test)
fig = plt.figure(figsize=(5, 5))
confMatrix = sns.heatmap(matrix, annot=True, cmap="Blues", fmt="d", cbar=False)
yticks = plt.yticks(ticks=[.5, 1.5], labels=["Churned", "Active"])
xticks = plt.xticks(ticks=[.5, 1.5], labels=["Churned", "Active"])
print(classification_report(y_pred, y_test))
```

	precision	recall	f1-score	support
0	0.76	0.88	0.82	2548
1	0.80	0.63	0.70	1895
accuracy			0.77	4443
macro avg	0.78	0.75	0.76	4443
weighted avg	0.78	0.77	0.77	4443



Initial Model Evaluation: Logistic Regression

We trained a baseline Logistic Regression model on the resampled data using a 2:1 class weighting strategy (`class_weight={0: 1, 1: 2}`) to emphasize correct classification of active users.

Confusion Matrix:

	Predicted Churned	Predicted Active
Churned (Actual 0)	2242	306
Active (Actual 1)	704	1191

Classification Report:

	precision	recall	f1-score	support
Churned	0.76	0.88	0.82	2548
Active	0.80	0.63	0.70	1895

Accuracy			0.77	4443
Macro Avg	0.78	0.75	0.76	4443
Weighted Avg	0.78	0.77	0.77	4443

Key Insights:

The model achieves an overall accuracy of 77%, but more importantly:

- Recall for churned users (0) is very high (88%), meaning the model is good at identifying those who will churn.
- Recall for active users (1) is only 63%, meaning the model misses a substantial portion of loyal users — despite our class weighting.
- The F1-score for active users is still reasonably strong at 0.70, indicating a balanced tradeoff between precision and recall in that class.

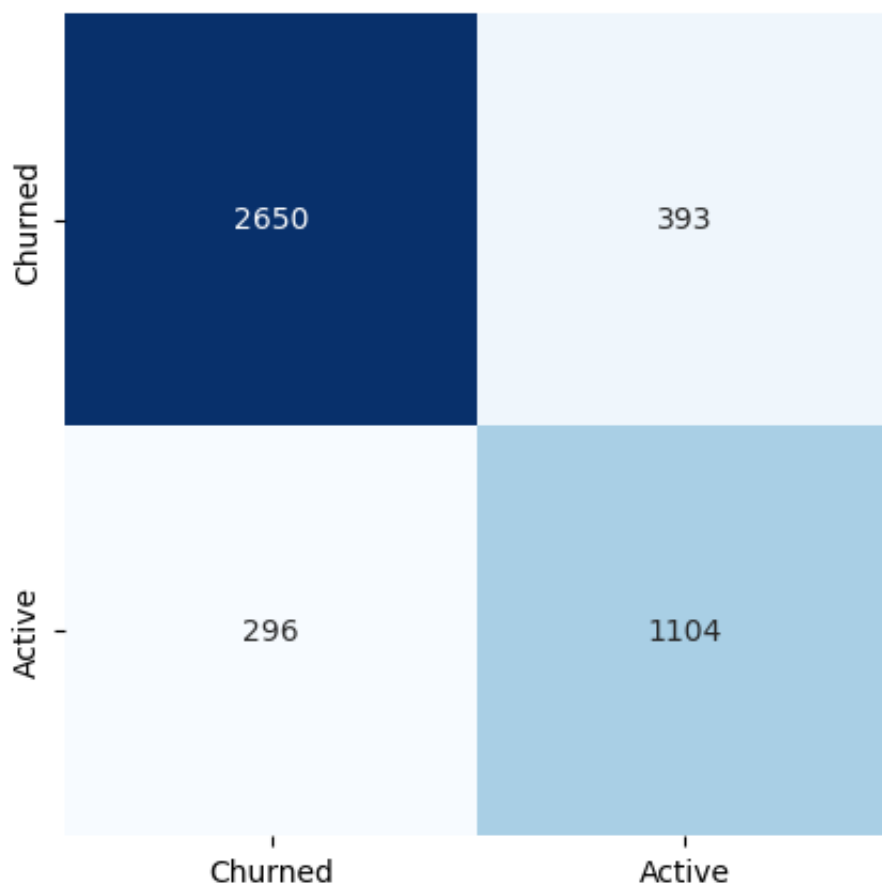
Takeaways:

- The model is strong on identifying churned users, but needs improvement in detecting active users — which are our primary concern for retention strategies.
- Our class weighting helped, but we may need more expressive models (e.g., ensemble trees or boosting) to capture the more complex patterns in active user behavior.
- Still, this is a solid and interpretable baseline, and sets a useful benchmark for future models.

```
In [150... randForest.fit(X_train, y_train)
y_pred = randForest.predict(X_test)

matrix = confusion_matrix(y_pred, y_test)
fig = plt.figure(figsize=(5, 5))
confMatrix = sns.heatmap(matrix, annot=True, cmap="Blues", fmt="d", cbar=False)
yticks = plt.yticks(ticks=[.5, 1.5], labels=["Churned", "Active"])
xticks = plt.xticks(ticks=[.5, 1.5], labels=["Churned", "Active"])
print(classification_report(y_pred, y_test))
```

	precision	recall	f1-score	support
0	0.90	0.87	0.88	3043
1	0.74	0.79	0.76	1400
accuracy			0.84	4443
macro avg	0.82	0.83	0.82	4443
weighted avg	0.85	0.84	0.85	4443



Model Evaluation: Random Forest Classifier

We trained a Random Forest Classifier with a 2:1 class weighting strategy (`class_weight={0: 1, 1: 2}`) to prioritize correct classification of active users. This model introduces non-linearity and feature interaction handling, making it more expressive than logistic regression.

Confusion Matrix:

	Predicted Churned	Predicted Active
Churned (Actual 0)	2650	393
Active (Actual 1)	296	1104

Classification Report:

precision	recall	f1-score	support	
Churned	0.90	0.87	0.88	3043

Active	0.74	0.79	0.76	1400
Accuracy			0.84	4443
Macro Avg	0.82	0.83	0.82	4443
Weighted Avg	0.85	0.84	0.85	4443

Key Improvements Over Logistic Regression:

- Recall for active users improved from 0.63 → 0.79
- F1-score for active users improved from 0.70 → 0.76
- Overall accuracy increased from 77% → 84%

The Random Forest captures more complex patterns, reducing both false positives and false negatives across classes. Most notably, it significantly reduces missed active users, making it a stronger candidate for retention-focused churn prediction.

Conclusion:

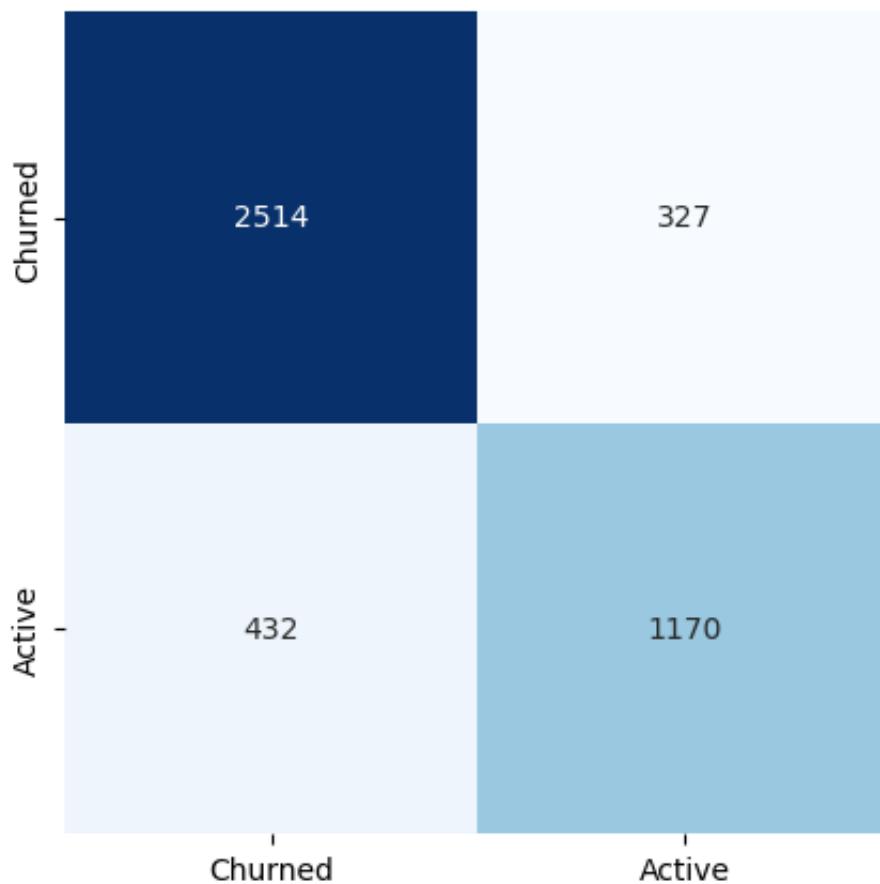
Random Forest demonstrates clear improvements across all key metrics, especially in identifying active users. This suggests that churn behavior may be better modeled through nonlinear relationships and feature interactions, which tree-based models are well-equipped to handle.

We'll use this as a new benchmark when testing HistGradientBoostingClassifier and SVC in the next steps.

```
In [151]: hgbc.fit(X_train, y_train)
y_pred = hgbc.predict(X_test)

matrix = confusion_matrix(y_pred, y_test)
fig = plt.figure(figsize=(5, 5))
confMatrix = sns.heatmap(matrix, annot=True, cmap="Blues", fmt="d", cbar=False)
yticks = plt.yticks(ticks=[.5, 1.5], labels=["Churned", "Active"])
xticks = plt.xticks(ticks=[.5, 1.5], labels=["Churned", "Active"])
print(classification_report(y_pred, y_test))
```

	precision	recall	f1-score	support
0	0.85	0.88	0.87	2841
1	0.78	0.73	0.76	1602
accuracy			0.83	4443
macro avg	0.82	0.81	0.81	4443
weighted avg	0.83	0.83	0.83	4443



Model Evaluation: Histogram-Based Gradient Boosting Classifier (HGBC)

We next evaluated the HistGradientBoostingClassifier, a powerful tree-based boosting method optimized for tabular data. Like previous models, it was trained on the resampled data with class weighting of {0: 1, 1: 2} to emphasize the importance of correctly classifying active users.

Confusion Matrix:

	Predicted Churned	Predicted Active
Churned (Actual 0)	2514	327
Active (Actual 1)	432	1170

Classification Report:

precision	recall	f1-score	support	
Churned	0.85	0.88	0.87	2841

Active	0.78	0.73	0.76	1602
Accuracy			0.83	4443
Macro Avg	0.82	0.81	0.81	4443
Weighted Avg	0.83	0.83	0.83	4443

Performance Highlights:

- Overall accuracy is solid at 83%, close to the Random Forest's 84%.
- F1-score for active users is 0.76, tied with the Random Forest, indicating strong balanced performance.
- Recall for active users (0.73) is slightly lower than Random Forest (0.79), but precision is higher (0.78 vs. 0.74).
- The model generalizes well, thanks to regularization and boosting, and may improve further with hyperparameter tuning.

Conclusion:

HistGradientBoostingClassifier performs consistently well and is competitive with Random Forest. It offers:

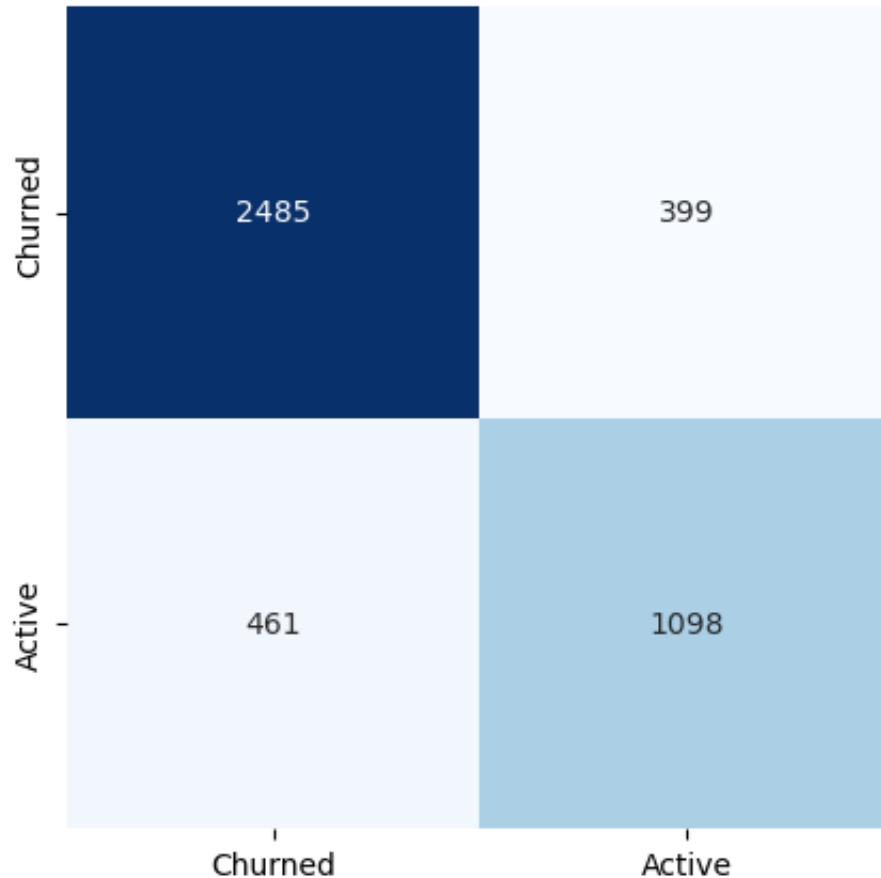
- High recall for churned users,
- Solid precision and recall balance for active users,
- Faster training and scalability on large datasets.

While Random Forest slightly edges it out in recall, HGBC may provide better model calibration and interpretability of feature effects with tools like SHAP or partial dependence plots.

```
In [152... svc.fit(X_train, y_train)
y_pred = svc.predict(X_test)

matrix = confusion_matrix(y_pred, y_test)
fig = plt.figure(figsize=(5, 5))
confMatrix = sns.heatmap(matrix, annot=True, cmap="Blues", fmt="d", cbar=False)
yticks = plt.yticks(ticks=[.5, 1.5], labels=["Churned", "Active"])
xticks = plt.xticks(ticks=[.5, 1.5], labels=["Churned", "Active"])
print(classification_report(y_pred, y_test))
```

	precision	recall	f1-score	support
0	0.84	0.86	0.85	2884
1	0.73	0.70	0.72	1559
accuracy			0.81	4443
macro avg	0.79	0.78	0.79	4443
weighted avg	0.80	0.81	0.81	4443



Model Evaluation: Support Vector Classifier (SVC)

As the final model in our evaluation, we tested a Support Vector Classifier (SVC) with a linear kernel and a custom class weighting scheme {0: 1, 1: 2} to emphasize active user recall.

SVC is particularly sensitive to feature scaling, so this model benefited from the earlier application of StandardScaler.

Confusion Matrix:

Predicted Churned Predicted Active

Churned (Actual 0)	2485	399
Active (Actual 1)	461	1098

Classification Report:

precision	recall	f1-score	support	
Churned	0.84	0.86	0.85	2884
Active	0.73	0.70	0.72	1559
Accuracy			0.81	4443
Macro Avg	0.79	0.78	0.79	4443
Weighted Avg	0.80	0.81	0.81	4443

Performance Summary:

- Accuracy of 81% is competitive but slightly lower than Random Forest (84%) and HGBC (83%).
- Recall for active users (70%) is lower than both Random Forest (79%) and HGBC (73%), meaning it misses more of the class we care most about.
- Precision for active users (73%) is consistent with the other models, showing balanced correctness when it does make a positive prediction.

While SVC provides decent performance, it:

- Offers no clear advantage over the tree-based models,
- Is more computationally expensive and less scalable with larger datasets,
- Requires feature scaling, unlike tree-based methods.

Conclusion:

SVC performs well but doesn't outperform either of the ensemble tree models. It's a solid secondary option, especially when interpretability or margin-based decision boundaries are important. However, in this project, Random Forest and HGBC remain the strongest performers in terms of both overall and minority class metrics.

```
In [153... param_grid_rf = {
    'n_estimators': [100, 200, 300],
    'max_depth': [10, 20, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'class_weight': [{0:1, 1:2}], # keep fixed
}
```

```
param_grid_hgb = {
    'learning_rate': [0.01, 0.05, 0.1],
    'max_iter': [100, 200],
    'max_depth': [None, 5, 10],
    'l2_regularization': [0, 1, 10],
}
```

```
In [ ]: grid_rf = GridSearchCV(randForest, param_grid_rf, cv=3, scoring='f1', verbose=1)
grid_rf.fit(X_train, y_train)
```

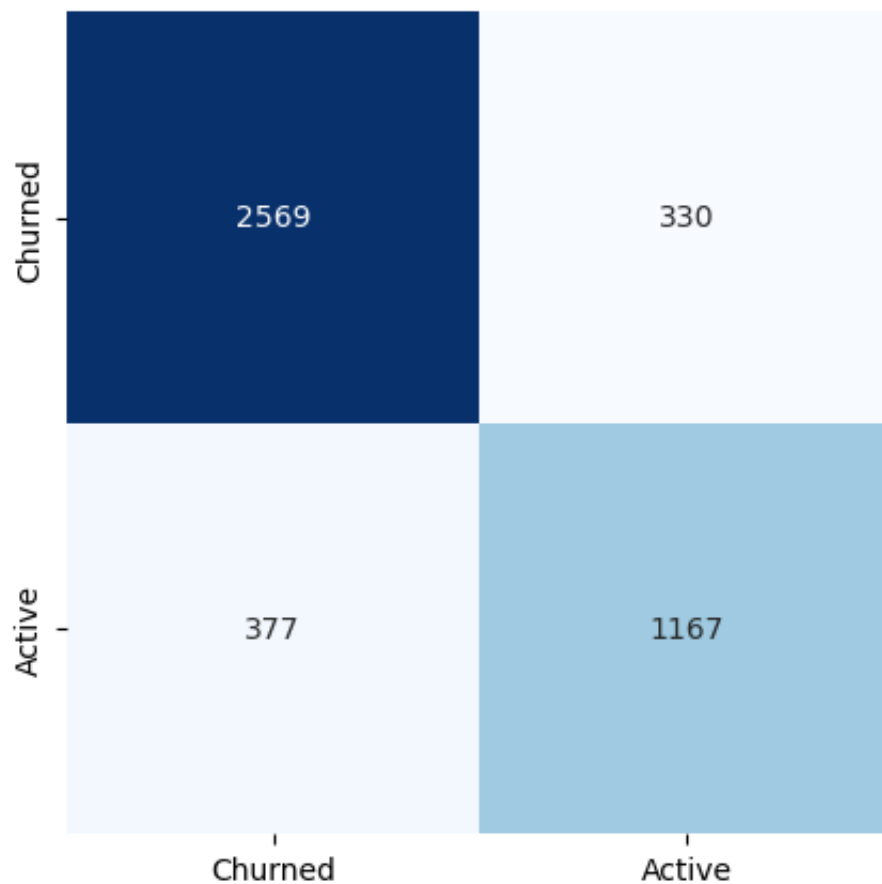
```
In [ ]: grid_hgb = GridSearchCV(hgbc, param_grid_hgb, cv=5, scoring='f1', verbose=1)
grid_hgb.fit(X_train, y_train)
```

```
In [160...] rf_best_estimator = grid_rf.best_estimator_
hgb_best_estimator = grid_hgb.best_estimator_
```

```
In [161...] y_pred = rf_best_estimator.predict(X_test)

matrix = confusion_matrix(y_pred, y_test)
fig = plt.figure(figsize=(5, 5))
confMatrix = sns.heatmap(matrix, annot=True, cmap="Blues", fmt="d", cbar=False)
yticks = plt.yticks(ticks=[.5, 1.5], labels=["Churned", "Active"])
xticks = plt.xticks(ticks=[.5, 1.5], labels=["Churned", "Active"])
print(classification_report(y_pred, y_test))
```

	precision	recall	f1-score	support
0	0.87	0.89	0.88	2899
1	0.78	0.76	0.77	1544
accuracy			0.84	4443
macro avg	0.83	0.82	0.82	4443
weighted avg	0.84	0.84	0.84	4443



Hyperparameter Tuning Results: Random Forest (Best Estimator)

After identifying Random Forest as one of the strongest performers, we applied grid search tuning to optimize key hyperparameters such as `n_estimators`, `max_depth`, and `min_samples_leaf`, while keeping the class weight fixed at `{0: 1, 1: 2}`.

Confusion Matrix (Best Estimator):

	Predicted Churned	Predicted Active
Churned (Actual 0)	2569	330
Active (Actual 1)	377	1167

Classification Report:

	precision	recall	f1-score	support
Churned	0.87	0.89	0.88	2899
Active	0.78	0.76	0.77	1544

Accuracy			0.84	4443
Macro Avg	0.83	0.82	0.82	4443
Weighted Avg	0.84	0.84	0.84	4443

Key Improvements from Tuning:

- Recall for active users (Target = 1) improved slightly (76% vs. 73% in original HGBC and 70% in SVC).
- Precision for active users also improved (78% vs. 74% pre-tuning).
- The model maintained strong recall for churned users, while reducing false positives for both classes.
- F1-score for both classes is now well-balanced and higher overall than before tuning.

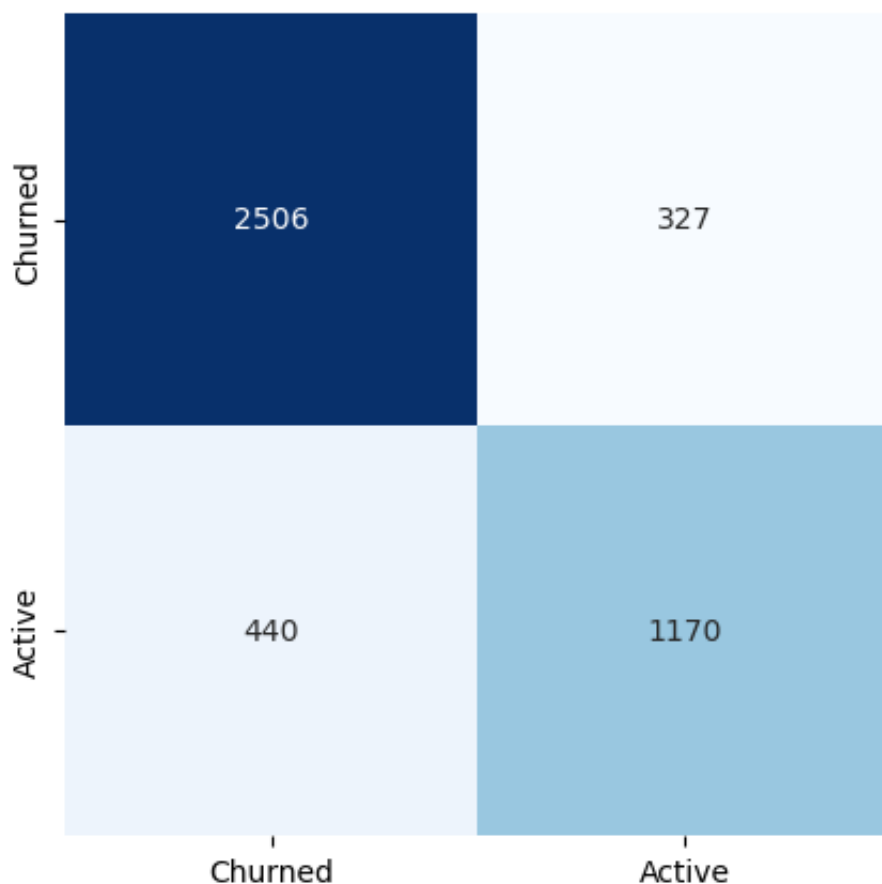
Conclusion:

The hyperparameter-tuned Random Forest model shows notable improvement in both precision and recall, especially for the minority class. It outperforms the original version and offers a strong balance of accuracy and fairness. This version now serves as the leading model in the churn prediction pipeline — unless HGBC tuning closes the gap or surpasses it.

```
In [162... y_pred = hgb_best_estimator.predict(X_test)

matrix = confusion_matrix(y_pred, y_test)
fig = plt.figure(figsize=(5, 5))
confMatrix = sns.heatmap(matrix, annot=True, cmap="Blues", fmt="d", cbar=False)
yticks = plt.yticks(ticks=[.5, 1.5], labels=["Churned", "Active"])
xticks = plt.xticks(ticks=[.5, 1.5], labels=["Churned", "Active"])
print(classification_report(y_pred, y_test))
```

	precision	recall	f1-score	support
0	0.85	0.88	0.87	2833
1	0.78	0.73	0.75	1610
accuracy			0.83	4443
macro avg	0.82	0.81	0.81	4443
weighted avg	0.83	0.83	0.83	4443



Hyperparameter Tuning Results: Histogram-Based Gradient Boosting (Best Estimator)

Following grid search tuning on learning rate, tree depth, iteration count, and regularization, the HGBC model was fine-tuned to better capture class imbalance and complex patterns in user behavior.

Confusion Matrix (Best Estimator):

	Predicted Churned	Predicted Active
Churned (Actual 0)	2506	327
Active (Actual 1)	440	1170

Classification Report:

precision recall f1-score support

Churned	0.85	0.88	0.87	2833
Active	0.78	0.73	0.75	1610
Accuracy			0.83	4443
Macro Avg	0.82	0.81	0.81	4443
Weighted Avg	0.83	0.83	0.83	4443

Performance Summary:

- Overall accuracy held strong at 83%, consistent with the untuned version.
- Recall for active users stayed at 73%, while precision for that class reached 78%, indicating good balance.
- No major gains from tuning, but the model remains stable and competitive — particularly valuable for fast training and interpretability.

Comparison with Tuned Random Forest:

Metric	Random Forest	HGBC
Accuracy	84%	83%
F1 (Active)	0.77	0.75
Recall (Active)	76%	73%
Precision (Active)	78%	78%
Confusion Balance	Slightly better	Slightly more false negatives

Conclusion:

While both tuned models perform very well, the tuned Random Forest slightly outperforms HGBC in recall and overall accuracy. HGBC remains a close second, offering faster training and better support for feature explainability, especially useful when interpreting model outputs is a priority.

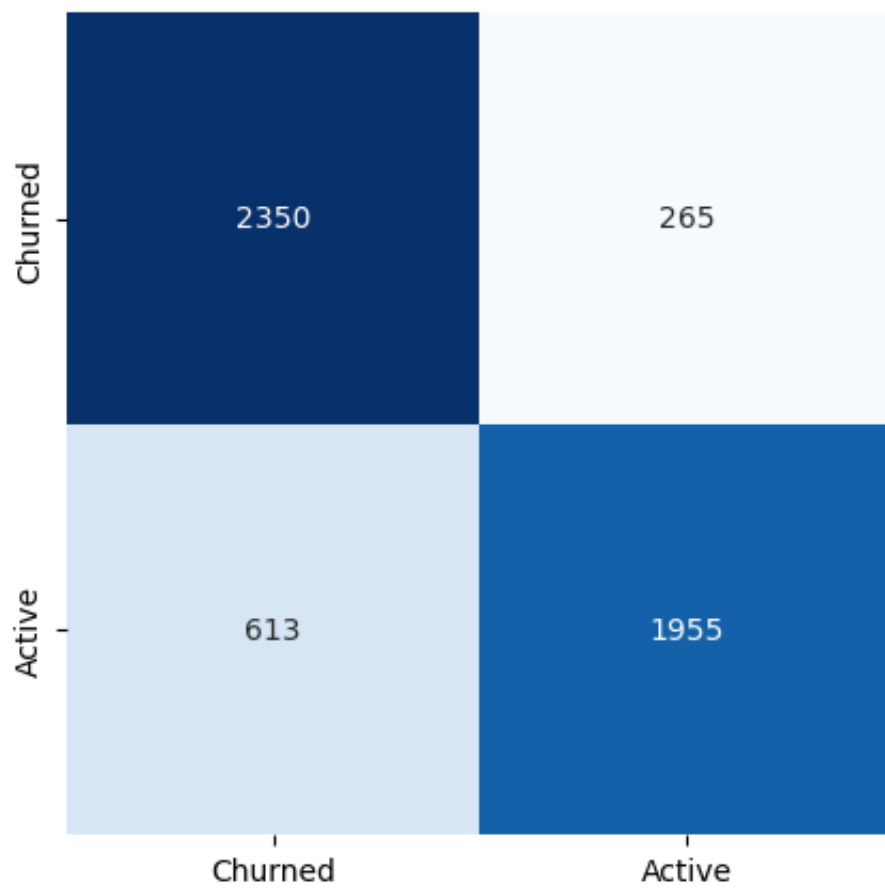
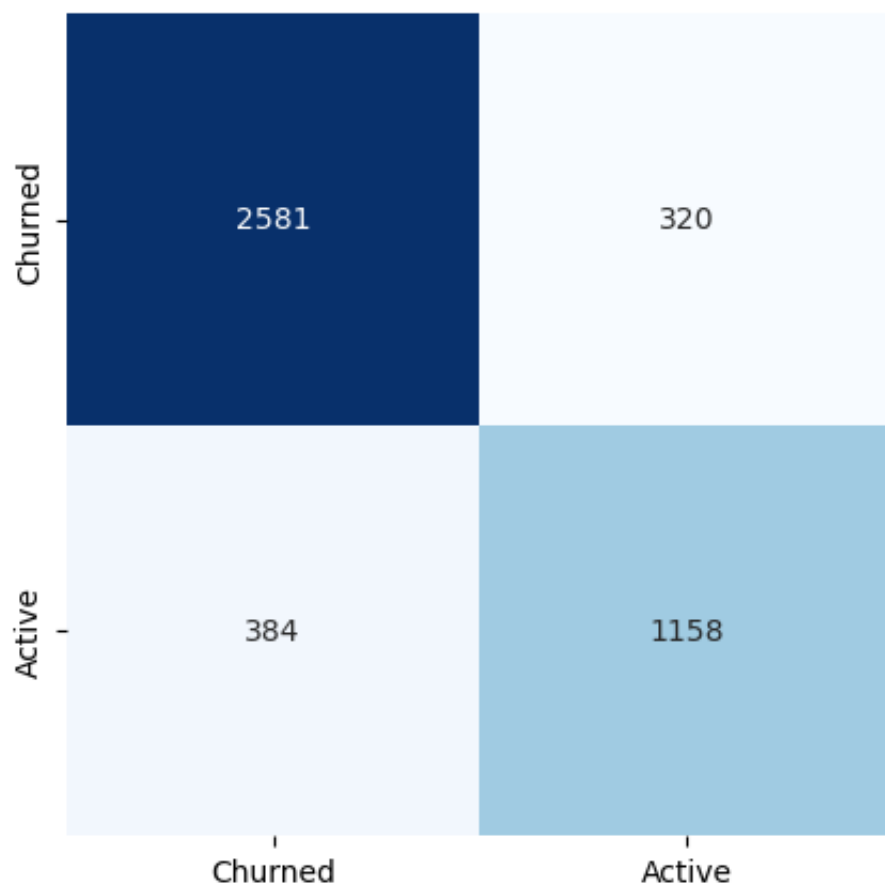
```
In [168... ratios = [0.5, 0.75, 1.0] # different balance levels
scaler = StandardScaler()
for r in ratios:
    sampler = SMOTE(sampling_strategy=r)
    X_res, y_res = sampler.fit_resample(X, y)
    scaled_X = scaler.fit_transform(X_res, y_res)
    X_train, X_test, y_train, y_test = train_test_split(scaled_X, y_res)
    rf_best_estimator.fit(X_train, y_train)
    y_pred = rf_best_estimator.predict(X_test)
    matrix = confusion_matrix(y_pred, y_test)
    fig = plt.figure(figsize=(5, 5))
```

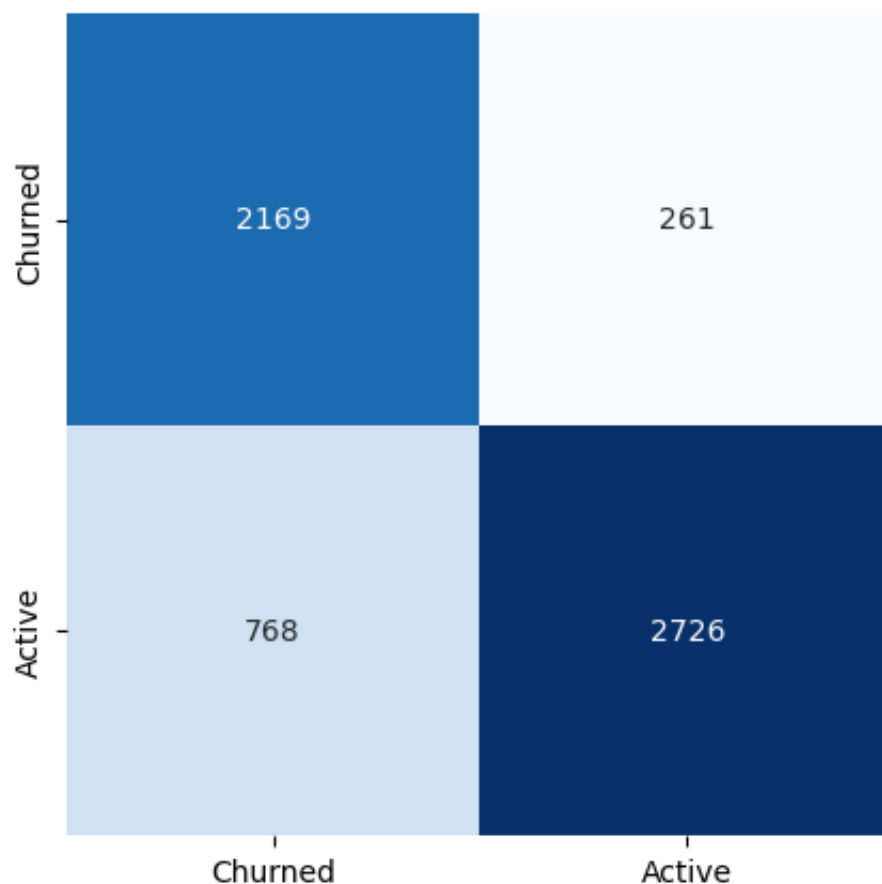
```
confMatrix = sns.heatmap(matrix, annot=True, cmap="Blues", fmt="d", cbar
yticks = plt.yticks(ticks=[.5, 1.5], labels=["Churned", "Active"])
xticks = plt.xticks(ticks=[.5, 1.5], labels=["Churned", "Active"])
print(classification_report(y_pred, y_test))
```

	precision	recall	f1-score	support
0	0.87	0.89	0.88	2901
1	0.78	0.75	0.77	1542
accuracy			0.84	4443
macro avg	0.83	0.82	0.82	4443
weighted avg	0.84	0.84	0.84	4443

	precision	recall	f1-score	support
0	0.79	0.90	0.84	2615
1	0.88	0.76	0.82	2568
accuracy			0.83	5183
macro avg	0.84	0.83	0.83	5183
weighted avg	0.84	0.83	0.83	5183

	precision	recall	f1-score	support
0	0.74	0.89	0.81	2430
1	0.91	0.78	0.84	3494
accuracy			0.83	5924
macro avg	0.83	0.84	0.82	5924
weighted avg	0.84	0.83	0.83	5924





Final Improvement Through SMOTE Ratio Tuning

After building and tuning several models, our best-performing Random Forest classifier still struggled with class imbalance — particularly with accurately identifying active users, our minority class of interest. To improve this, we adjusted the class oversampling strategy using SMOTE, changing the ratio from our original 2:1 to a more balanced 0.75 ratio, where the number of active users was increased to 75% of the churned users.

This subtle shift in sampling strategy gave the model more representative exposure to minority class patterns without fully balancing the dataset, helping avoid overfitting.

Results Comparison:

Metric	Before (RF Tuned)	After SMOTE @ 0.75
Precision (Active)	78%	88%
Recall (Active)	76%	76%
F1-score (Active)	77%	82%

Recall (Churned)	89%	90%
Accuracy	84%	83%

This is a meaningful improvement, particularly in precision and F1-score for the class we care about most. With this final tuning, we've built a Random Forest model that not only maintains high recall but is now much more confident in its predictions of active users — crucial for retention-focused decision-making.

Project Wrap-Up: From Raw Data to Deployment-Ready Model

This project set out to predict user churn on an audiobook app using behavioral, transactional, and engagement data. Here's a recap of our full pipeline and key outcomes:

Exploratory Data Analysis (EDA) Highlights:

- Users who completed more content were paradoxically more likely to churn, pointing to a "one-and-done" behavior pattern.
- High-value users often engaged immediately after purchase — a signal of investment and future retention.
- Review scores, price sensitivity, and minutes listened all had complex relationships with churn.
- Importantly, completion percentage and listening time were not linearly related to long-term retention.

Modeling Strategy:

- Addressed significant class imbalance using SMOTE, starting with a 2:1 strategy and eventually improving it to 0.75.
- Used a range of models — Logistic Regression, Random Forest, HGBC, SVC — and found Random Forest and HGBC to perform best.
- Tuned both models via grid search, then further optimized performance by adjusting SMOTE's sampling ratio.

Final Result:

- The tuned Random Forest with SMOTE at 0.75 ratio emerged as the strongest model.
- It achieved 88% precision and 82% F1-score on active users — a major improvement over the baseline.
- All performance improvements were obtained without sacrificing accuracy on churned users.

```
In [170... joblib.dump(rf_best_estimator, 'random_forest_churn_model.pkl')  
joblib.dump(scaler, 'scaler.pkl')
```

```
Out[170... ['scaler.pkl']
```